

TUNE — Le Serveur Musical Open Source qui Vise l'Excellence Audiophile

Dossier de presse — Avril 2026

MozAlk Labs — L'essentiel

Bertrand Clech — Fondateur & Lead Developer

Ingenieur EPFL (Systemes de Communication — convergence IT/Telecom, 1995). Plus de 30 ans d'experience en ingenierie logicielle, telecom et systemes distribues. Audiophile et entrepreneur, passionne par le rapprochement entre l'audio haut de gamme et le logiciel moderne.

MozAlk Labs est une entreprise francaise dediee a la nouvelle generation de logiciels serveur musicaux pour audiophiles. Mission : offrir une lecture de qualite studio avec le confort du streaming moderne — sans compromis.

Fondateur	Bertrand Clech
Societe	MozAlk Labs
Site web	mozaiklabs.fr
Localisation	France
Produit	Tune — Serveur Musical Multi-room
Statut	Beta ouverte (v0.5.3, Avril 2026)
Communaute	Beta testeurs actifs via mozaiklabs.fr/forum

Equipement de reference :

- Micromega M-One (ampli/DAC/streamer)
- EverSolo DMP-A8 (streamer)
- Lindemann (streamer)
- Sonos (multi-room)

Services de Streaming : Tidal HiFi+, Qobuz Studio

Equipe :

- Bertrand — Architecture, backend (Python/FastAPI), iOS (Swift/SwiftUI), infrastructure
- Matteo — Frontend, ecommerce (React/Laravel)
- JP — Conseiller en architecture
- Freddy — Partenariats materiel HiFi (Belgique)
- Claude AI (Anthropic) — Developpement assiste par IA & prototypage rapide

En une phrase

Tune est un **serveur musical multi-room open source** qui unifie bibliotheques locales, partages reseau et 6 services de streaming avec une **lecture bit-perfect verifiee** vers les renderers DLNA/UPnP, AirPlay et les DAC USB — le tout pilotable depuis un iPad, un iPhone, un navigateur web ou une app Android.

Pourquoi Tune merite qu'on en parle

Le constat

Le marche du streaming audiophile est domine par des solutions fermees (Roon, BluOS, HEOS) ou limitees a un ecosysteme. L'audiophile qui possede un DAC haut de gamme, un NAS, des abonnements Tidal et Qobuz, et des appareils Apple et Android n'a pas de solution unifiee, ouverte et gratuite.

La reponse de Tune

- **Open source et gratuit** — pas d'abonnement, pas de licence, code source public sur GitHub
- **Bit-perfect verifie** — checksum MD5 de bout en bout, pastille verte dans l'interface quand le signal n'a subi aucune alteration
- **6 services de streaming** — Tidal HiFi+ (192/24), Qobuz Studio (192/24), YouTube Music, Spotify, Deezer, Amazon Music
- **Fonctionne partout** — Linux, macOS, Windows, iPadOS, iOS, Android, Docker, Raspberry Pi
- **Multi-room avec synchronisation** — groupement de zones, compensation de delai par appareil
- **DSD natif** — passthrough DSF/DFF vers les renderers compatibles, sans conversion
- **Developpe avec l'IA** — Claude (Anthropic) est co-developpeur, accelerant la vitesse d'iteration d'un facteur 10

Ce qui le differencie de Roon

	Tune	Roon
Prix	Gratuit, open source	14,99 \$/mois ou 829 \$ a vie
Code source	Public (GitHub)	Ferme
DSD natif	Oui (passthrough)	Oui
Bit-perfect verifie	Oui (checksum MD5)	Non (signal path indicatif)
Chemin du signal	Oui (inspire Roon)	Oui (pionniers)
Services streaming	Tidal, Qobuz + 4 autres	Tidal, Qobuz uniquement
iPad comme serveur	Oui (mode autonome)	Non
Multi-room	Oui (DLNA + AirPlay)	Oui (RAAT propriétaire)
Auto-heberge	Oui	Oui (Roon Core)
DSP / Convolution	Oui (FFmpeg)	Oui

Comment ca marche — Pour le lecteur

Le plus simple : un iPad et un DAC

L'iPad fait tourner Tune en mode serveur. Il scanne la musique locale, se connecte a Tidal et Qobuz, et envoie l'audio en DLNA a votre DAC. Pas de PC, pas de NAS — juste un iPad et votre systeme audio.

L'installation audiophile : serveur Linux + telecommande

Un petit PC (Intel NUC, Raspberry Pi, ou meme un NAS Docker) fait tourner le serveur. Il scanne votre NAS, gere les abonnements streaming, et envoie l'audio vers un ou plusieurs renderers DLNA/AirPlay. Vous controlez tout depuis un iPad, un

iPhone, ou un navigateur web.

Le multi-room

Plusieurs zones, chacune avec sa file d'attente et son volume. Groupez-les pour une écoute synchronisée dans toute la maison. Mix DLNA haute fidélité dans le salon + Sonos dans la cuisine + AirPlay dans la chambre.

1b. Cas d'Usage — Comment Tune s'adapte à votre Installation

Scénario 1 : iPad seul (autonome)

```
graph LR
    IPAD["📱 iPad<br/>Tune Mode Serveur"] -->|DLNA/UPnP| DAC["🔊 DAC Haut de Gamme"]
    IPAD -->|Tidal / Qobuz| DAC
    style IPAD fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style DAC fill:#ff6b35,stroke:#fff,color:#fff
```

- L'iPad fait tourner Tune en **mode serveur** (moteur embarqué)
- Scanne la musique locale (stockage iPad + bibliothèque Apple Music)
- Se connecte aux services de streaming (Tidal, Qobuz)
- Envoie l'audio via **DLNA/UPnP** directement à votre DAC
- **Multi-room** : découvre plusieurs rendereurs DLNA, peut grouper des zones
- **Ideal pour** : installation simple, audiophile nomade

Scénario 1b : iPhone seul (audiophile portable)

```
graph LR
    IP["📱 iPhone<br/>Tune Mode Serveur<br/>+ Zone Locale"] -->|DLNA/UPnP| DAC["🔊 DAC Haut de Gamme"]
    IP -->|Bluetooth| BT["🎧 Casque"]
    IP -->|AirPlay| AP["🔊 Enceinte AirPlay"]
    style IP fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style DAC fill:#ff6b35,stroke:#fff,color:#fff
```

- L'iPhone fait tourner Tune en **mode serveur** (autonome)
- Streamer depuis Tidal/Qobuz, envoie vers DLNA, Bluetooth ou AirPlay
- **Ideal pour** : écoute nomade, contrôle DLNA rapide

Scénario 2 : Serveur Linux + iPad/iPhone en télécommande

```
graph LR
    IPAD["📱 iPad / iPhone<br/>Télécommande"] -->|REST API + WS| SRV["🖥️ Serveur Linux<br/>NAS · Tidal · Qobuz<br/>22 000+ pistes"]
    WEB["🌐 Navigateur web"] -->|REST API| SRV
    SRV -->|DLNA| DAC["🔊 DAC Haut de Gamme"]
    SRV -->|DLNA| SONOS["🔊 Sonos<br/>Pièce 2"]
    style SRV fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style DAC fill:#ff6b35,stroke:#fff,color:#fff
```

- Serveur Linux (Intel NUC, Raspberry Pi, ou tout PC) fait tourner **tune-server**

- Bibliotheque complete avec enrichissement metadonnees, playlists
- **Ideal pour** : installation audiophile serieuse, grande bibliotheque, multi-room

Scenario 3 : Serveur Linux + sorties multiples

```
graph LR
    CTRL["📱 Tout appareil<br/>de controle"] -->|API| SRV["🖥️ Serveur Linux<br/>Sync multi-room"]
    SRV -->|DLNA| TOT["🔊 DAC Haut de Gamme<br/>Salon"]
    SRV -->|DLNA| MICRO["🔊 Micromega<br/>Bureau"]
    SRV -->|AirPlay| AP["🔊 AirPlay<br/>Cuisine"]
    SRV -->|USB| USB["🎧 DAC USB<br/>Casque"]
    style SRV fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style TOT fill:#ff6b35,stroke:#fff,color:#fff
```

- Plusieurs zones simultanees, chacune avec sa file d'attente et son volume
- Zones groupables pour une lecture synchronisee (multi-room)
- Mix de sorties DLNA, AirPlay et DAC USB
- **Ideal pour** : sonorisation de toute la maison

Scenario 4 : Mac de bureau (tout-en-un)

```
graph LR
    MAC["🖥️ Mac<br/>tune-server + Tune.app"] -->|USB| DAC["🔊 DAC Haut de Gamme<br/>Entree USB"]
    style MAC fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style DAC fill:#ff6b35,stroke:#fff,color:#fff
```

- tune-server + Tune.app natif, sortie USB directe vers le DAC
- **Ideal pour** : audiophile de bureau, ecoute au casque

Scenario 5 : Raspberry Pi (audiophile embarque)

```
graph LR
    PHONE["📱 Telephone / Web"] -->|WiFi| RPI["🍓 Raspberry Pi 5<br/>tune-server · headless<br/>SSD + NAS"]
    RPI -->|USB| DAC["🔊 DAC Haut de Gamme<br/>Entree USB"]
    style RPI fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style DAC fill:#ff6b35,stroke:#fff,color:#fff
```

- Streamer dedie pour moins de 100 euros
- Sortie USB bit-perfect vers le DAC, controle depuis n'importe quel appareil
- **Ideal pour** : streamer dedie ultra-economique

Scenario 6 : Docker sur NAS

```
graph LR
    NAS["🖥️ Synology NAS<br/>Docker · tune-server<br/>musique sur NAS"] -->|DLNA| DAC["🔊 DAC Haut de Gamme"]
    style NAS fill:#1a1a2e,stroke:#ff6b35,color:#fff
    style DAC fill:#ff6b35,stroke:#fff,color:#fff
```

- Tourne dans Docker sur Synology, QNAP, Unraid
- Bibliotheque deja sur le NAS — zero copie
- **Ideal pour** : proprietaires de NAS, zero materiel supplementaire

Comparaison rapide

Installation	Materiel	Multi-room	Chemin Audio	Complexite
iPad seul	iPad	Oui (DLNA)	iPad -> DLNA -> DAC	Facile
Linux + telecommande	Serveur + iPad	Oui	Serveur -> DLNA -> DAC	Moyen
Linux + multi	Serveur + tout	Oui (sync)	Sorties multiples	Avance
Mac de bureau	Mac	Optionnel	Mac -> USB -> DAC	Facile
Raspberry Pi	RPi + SSD	Optionnel	RPi -> USB -> DAC	Moyen
Docker / NAS	NAS	Oui	NAS -> DLNA -> DAC	Moyen

Architecture Technique — Pour les curieux

Stack Serveur (Linux / macOS / Windows)

Couche	Technologie	Role
Langage	Python 3.11+ (async)	Coeur serveur
API	FastAPI + Uvicorn	106+ endpoints REST + WebSocket
Base de donnees	SQLite / PostgreSQL (double moteur)	Bibliotheque, playlists, zones
Pipeline Audio	FFmpeg	Decodage, transcodage, resampling
DLNA/UPnP	async-upnp-client	Controle renderer, decouverte SSDP
AirPlay	pyatv	Streaming vers appareils Apple
Sortie Locale	sounddevice + numpy	DAC USB / carte son
Metadonnees	mutagen + musicbrainzngs	Lecture/ecriture de tags, enrichissement

Apps natives (iPadOS / iOS / macOS)

Couche	Technologie
Langage	Swift 6.0 (concurrency stricte)
UI	SwiftUI
DLNA	XMLParser + URLSession natifs
Streaming	Swift natif (sans dependances)

App Flutter (Android / iOS)

Couche	Technologie
Langage	Dart 3.11+
Audio	just_audio
Serveur embarque	Shelf + shelf_router

Client Web

Couche	Technologie
Langage	TypeScript 5.7+
Framework	Svelte 5 (runes)
Design	Responsive 3 breakpoints (bureau / tablette / mobile)
Langues	8 langues (FR, EN, DE, ES, IT, ZH, KO, JA)

Chemin du Signal — Ce qui interesse l'audiophile

Strategies de Lecture

Strategie	Quand	CPU	Qualite
Passthrough URL Directe	Streaming -> DLNA	Zero	Bit-perfect
Passthrough DSD Natif	DSF/DFF -> renderer compatible	Zero	Bit-perfect
Passthrough Fichier	FLAC local -> renderer compatible	Minimal	Bit-perfect
Transcodage FFmpeg	Incompatibilite de format	Moyen	Transparent

Le principe : **ne jamais toucher au signal si ce n'est pas necessaire**. Quand un fichier FLAC est lu sur un DAC compatible FLAC, Tune transmet le flux tel quel — zero decodage, zero re-encodage. Le checksum MD5 le prouve.

Affichage du Chemin du Signal (inspire de Roon)

```
Source : Qobuz FLAC 96/24
-> Transport : Passthrough URL Directe
-> Renderer : DAC Haut de Gamme (DLNA)
-> Horloge : Interne (renderer)
-> Traitement : Aucun (bit-perfect)
-> Sortie : 96kHz / 24-bit / 2ch
```

Pastille verte dans la barre de transport = bit-perfect. Jaune = transcoding applique (avec le detail du pourquoi).

Formats Supportes

Format	Resolution Max	DSD	Gapless
FLAC	192 kHz / 24-bit	—	Oui

WAV	192 kHz / 32-bit	—	Oui
ALAC	192 kHz / 24-bit	—	Oui
DSD (DSF/DFF)	DSD128 (5,6 MHz)	Natif	Oui
DSD Fallback	176,4 kHz / 24-bit PCM	Converti	Oui
AAC/MP3/OGG	48 kHz / 16-bit	—	Oui

Qualite des Services de Streaming

Service	Qualite Max	Format	Resolution
Tidal	HI_RES_LOSSLESS	FLAC	192 kHz / 24-bit
Qobuz	Studio Ultra	FLAC	192 kHz / 24-bit
Amazon Music	ULTRA_HD	FLAC	96 kHz / 24-bit
Deezer	HiFi	FLAC	44,1 kHz / 16-bit
Spotify	Premium	OGG 320k	Lossy
YouTube	Meilleur disponible	AAC/OPUS	Variable

Excellence Audio — Les fonctionnalites cles (v0.5.3)

Verification Bit-Perfect

- Checksum MD5 de bout en bout (fichier source -> entree renderer)
- Comparaison de hash : `source_hash == output_hash` verifie en passthrough
- Indicateur visuel : pastille verte "Bit-Perfect" ou jaune "Transcode"
- Log complet des decisions du pipeline

Resampling Avance

- Politique configurable : `auto` , `never` , `integer_ratio` (preferer 44,1 -> 88,2 -> 176,4 kHz)
- Preference de ratio entier pour une conversion audiophile
- Resampler SoX via FFmpeg en fallback

Gestion des Buffers

- Taille de buffer configurable par sortie
- Mode basse latence (10ms) pour DAC USB local
- Mode buffer large (200ms) pour les reseaux difficiles

Support USB Audio Class

- **Mode exclusif** : bypass du mixer OS (PulseAudio/PipeWire) pour une sortie bit-perfect
- Passthrough DSD natif vers les renderers compatibles

Correction Acoustique / DSP

- Chaine de filtres DSP : tout filtre FFmpeg (egaliseur, basses, aigus, compresseur...)
- **Convolution** : import de fichiers de reponse impulsionnelle (Dirac Live, REW, Audiolens)
- Mode bypass pour les puristes (DSP desactive par default — zero traitement)

Metadata Readonly (v0.5.3 — nouveau)

- Mode lecture seule : Tune ne modifie jamais les tags de vos fichiers audio
 - Enrichissement automatique des images d'artistes via Discogs (sans toucher aux fichiers)
-

Architecture Multi-Room

```
Zone 1 : "Salon" (DLNA -> DAC Haut de Gamme)
|-- File d'attente : [Piste A, Piste B, Piste C]
|-- Volume : 0.65
+-- Etat : En lecture (position : 2:34)
```

```
Zone 2 : "Bureau" (DLNA -> Micromega M-One)
|-- File d'attente : [Piste X, Piste Y]
|-- Volume : 0.40
+-- Etat : En pause
```

```
Groupe "Toute la Maison" (leader : Zone 1)
|-- Zone 1 (leader, delai sync : 0ms)
+-- Zone 2 (follower, delai sync : +150ms)
```

- Polling adaptatif : 1s en lecture, 10s au repos
 - Seuil de detection de derive : 500ms
 - Compensation du delai par zone
 - Demarrage echelonne pour les renderers reseau
-

Diagrammes d'Architecture

Topologie Reseau

```
graph TD
    subgraph Server["🖥️ Tune Server (Linux/Mac)"]
        LIB["📖 Bibliotheque<br/>22 000+ pistes"]
        STR["🎵 Streaming<br/>Tidal · Qobuz"]
        DB["🗄️ PostgreSQL"]
    end

    subgraph Outputs["🔊 Sorties Audio"]
        TOT["DAC Haut de Gamme<br/>DLNA/USB"]
        AIR["Enceintes AirPlay"]
        SON["Sonos · DLNA"]
    end

    subgraph Clients["👤 Clients de Controle"]
        IPAD["iPad<br/>SwiftUI"]
        IPHONE["iPhone<br/>Telecommande"]
        WEB["Web<br/>Svelte 5"]
        MAC["macOS<br/>SwiftUI"]
    end
```



```
Server -->|"DLNA/UPnP<br/>Audio HTTP :8080"| Outputs
Server -->|"REST API + WebSocket<br/>:8888"| Clients
```

```
style Server fill:#1a1a2e,stroke:#ff6b35,color:#fff
style Outputs fill:#1e1e38,stroke:#64b5f6,color:#fff
style Clients fill:#1e1e38,stroke:#81c784,color:#fff
style TOT fill:#ff6b35,stroke:#fff,color:#fff
```

Feuille de Route

gantt

title Feuille de Route Tune Server – v1.0.0 cible : Mai 2026

dateFormat YYYY-MM-DD

axisFormat %d %b

section v0.5.3 (fait)

Profils & Favoris :done, 2026-04-01, 2026-04-05

Gestionnaire Playlists :done, 2026-04-01, 2026-04-05

Support PostgreSQL :done, 2026-04-01, 2026-04-05

Signal path & bit-perfect :done, 2026-04-05, 2026-04-07

Metadata readonly & enrichissement :done, 2026-04-06, 2026-04-07

section v0.6.0 – Corrections & Tests

Beta test iPadOS/macOS :active, 2026-04-07, 2026-04-14

Corrections scanner & lecture :active, 2026-04-07, 2026-04-14

section v0.7.0 – Connecteurs

Spotify Connect (streaming complet) :2026-04-14, 2026-04-21

Deezer HiFi (streaming complet) :2026-04-14, 2026-04-21

Amazon Music HD (playlists + play) :2026-04-14, 2026-04-21

Integration Apple Music :2026-04-14, 2026-04-21

section v0.8.0 – Finitions

Gestionnaire Playlists v2 (sync) :2026-04-21, 2026-04-28

Retours beta testeurs final :2026-04-21, 2026-04-28

section v0.9.0 – Plateformes

Sortie Android (Play Store) :2026-04-21, 2026-04-28

Sortie iOS / macOS (App Store) :2026-04-21, 2026-04-28

Image Docker officielle :2026-04-21, 2026-04-28

section v1.0.0 – Sortie

Tests finaux & stabilisation :crit, 2026-04-28, 2026-05-05

v1.0.0 Sortie Publique :milestone, 2026-05-05, 0d

section Post v1.0 – Excellence Audio

Sortie RAVENNA / AES67 :2026-05-05, 2026-06-15

Correction acoustique / DSP avance :2026-06-01, 2026-07-15

Partenariats materiel :2026-05-15, 2026-07-30

Linux Embarque (Raspberry Pi) :2026-06-01, 2026-07-15

Statut Actuel (v0.5.3 — Avril 2026)

Fonctionnalite	Statut	Cible
Gestion de bibliotheque	Production	—
Tidal HiFi+	Streaming complet (FLAC 192/24)	—
Qobuz Studio	Streaming complet (FLAC 192/24)	—
YouTube Music	Streaming	—
Spotify	Apercu uniquement	v0.7.0
Deezer	Apercu uniquement	v0.7.0
Amazon Music	Recherche uniquement	v0.7.0
Apple Music	iPadOS uniquement	v0.7.0
Sortie DLNA/UPnP	Bit-perfect, DSD natif, gapless	—
Sortie AirPlay	Production	—
Multi-room	Groupement de zones + moteur de sync	—
Chemin du signal	Affichage complet + verification bit-perfect	—
DSP / Convolution	Chaine FFmpeg + reponse impulsionnelle	—
Client web	Responsive, 8 langues	—
iPadOS / iOS / macOS	TestFlight	App Store v1.0
Android (Flutter)	Beta	Play Store v1.0
RAVENNA / AES67	Planifie	Post v1.0
Docker	Planifie	v0.9.0

Angles pour un article

L'angle technique

Un serveur musical open source ecrit en Python qui rivalise avec Roon sur le chemin du signal bit-perfect — et le prouve avec un checksum MD5 de bout en bout.

L'angle democratisation

Un Raspberry Pi a 80 euros + Tune = un streamer audiophile bit-perfect. Pas besoin d'un appareil a 2 000 euros.

L'angle IA

Claude AI (Anthropic) est co-developpeur du projet. L'IA accelere le developpement d'un facteur 10 — un seul developpeur produit l'equivalent d'une equipe de 5.

L'angle open source vs. propriétaire

Face a Roon (829 \$ a vie), BluOS (ferme, limite aux appareils NAD/Bluesound), HEOS (Denon/Marantz uniquement), Tune est gratuit, ouvert, et fonctionne avec tout appareil DLNA.

L'angle francais

MozAlk Labs est une entreprise francaise, le fondateur est ingénieur EPFL. Le logiciel est developpe en France.

Pour tester

1. **Le plus rapide** : Telecharger sur mozaiklabs.fr/download (Linux, macOS, Windows)
 2. **Sur iPad/iPhone** : Rejoindre le beta test TestFlight (contacter Bertrand)
 3. **Interface web** : Demo accessible a `http://serveur:8888` apres installation
-

Contact & Ressources

- **Contact presse** : Bertrand Clech — bertrand@mozaiklabs.fr
 - **Site web** : mozaiklabs.fr
 - **Telechargements** : mozaiklabs.fr/download
 - **GitHub** : github.com/renesenses/tune-server-linux
 - **Forum Beta** : mozaiklabs.fr/forum
 - **Version actuelle** : v0.5.3 (Avril 2026)
-

Document prepare pour Alban Lamouroux — MozAlk Labs, Avril 2026